

13 abril 2026

Optimización (Búsquedas locales)

$$f: X \rightarrow \mathbb{R}$$

y lo que quiero es encontrar una x^* tal que

$$f(x^*) = \max_{x \in X} f(x)$$

$$f(x^*) = \min_{x \in X} f(x)$$

$$A = \begin{bmatrix} 0 & a_{12} & a_{1n} \\ a_{21} & & \\ \vdots & \ddots & \\ a_{n1} & & 0 \end{bmatrix}$$

lo que quiero es encontrar un circuito cerrado

$x =$ permutación de $[1, \dots, n]$
donde

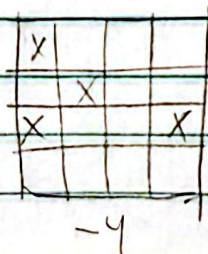
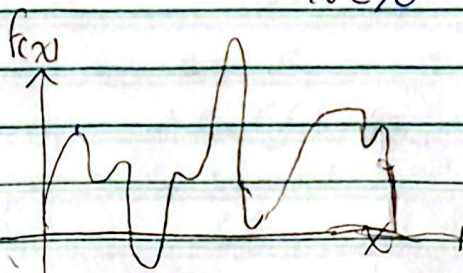
$$f(x) = \sum_{i=1}^{n-1} A_{x(i), x(i+1)} + A_{x(n), x(1)}$$

$$y \quad x^* = \min_x f(x)$$

$$f: X \rightarrow \mathbb{R}$$

Un máximo global es:

$$x^* = \arg \max_{x \in X} f(x) \quad (\text{puede haber uno o varios máximos globales})$$



Si existe una topología en X

$$x_1' = \lim_{\Delta \rightarrow \emptyset} x_1 - \Delta$$

$$x_2' = \lim_{\Delta \rightarrow \emptyset} x_1 + \Delta$$

un mínimo local
en;

$$x^* = \arg \max_{x \in \text{Vecinos}(x)} f(x)$$

- Máximo local es donde la derivada es 0 y segunda derivada es positiva o diferencia de mínimos locales donde la derivada es negativa.

- Genera una variable
- inicializa $x \in X$ de forma aleatoria
 $e_{max} \leftarrow f(x)$
para max_epoch iteraciones.

14 abril 2026

```
class P6Option():  
    def costo (self, x):  
        if not self.estado(x):  
            raise ValueError ( )  
        return f(x)
```

```
    def estado (self, x):  
        return True/False  
    def estado_aleatorio (self):  
        return estado
```

```
    def BusquedaAleatoria (pb, num_iter):  
        x = pb.estado_aleatorio()  
        I ← c = pb.costo (x)  
        for _ in range (num_iter):  
            otro = pb.estado_aleatorio()  
            c = otro = pb.costo (otro)  
            if costo < c:  
                x, c = otro, otro
```

```
    def vecinos (self, x):  
        return iterable con los estados vecinos  
        de x
```

```
    def descenso_colina (pb, num_iter, x=pb.estado_aleatorio()):  
        x = pb.estado_aleatorio()  
        c = pb.costo(x)  
        for _ in range (num_iter):  
            termina = False  
            for vecino in pb.vecinos(x):  
                c = vecino = pb.costo (vecino)  
                if c - vecino < c:
```

```

x, c = vecino, c - vecino
terminal = False
if terminal:
    break
return x, costo

```

- Metaheurística: Temple del metal

```

def vecino_aleatorio(self, x):
    return vecino(x)

```

```

def temple_simulado(pb, Tmax, calendario,
                    como se entra la temp, Tador, Tmin):

```

```

    x = pb - estado_aleatorio U
    c = pb - costo(x)
    T = Tmax

```

```

    while (T > Tmin):
        vecino = pb - vecino_aleatorio(x)
        c - vecino - pb, costo(vecino)

```

```

    incrementos → inc = c - c - vecino
    inverso
    - inc

```

```

    if inc ≥ 0 or random.random() ≤
        math.exp(inc / T):

```

```

        x, c = vecino, c - vecino

```

```

        T = calendario(T, i)

```

```

        i += 1

```

```

    return x, c

```

> Simulated annealing

6

def calcular_tmax(tmax, i):

$$\frac{T_{max}}{i+1}$$

$$\frac{T_{max}}{\sqrt{i+1}}$$

$$\frac{T_{max}}{\log(i+1)+1}$$

$$T_{max} e^{-k}$$

return $T_{max} / (\text{math.log}(i+1)+1)$

15 abril 2026

poblacion = $\{x_1, x_2, \dots, x_p\}$ $x_i \in X$

caract(x_i) = $[v_{i1}, v_{i2}, \dots, v_{in}]$ ADN

adaptacion: $X \rightarrow \mathbb{R}$ adaptacion(x_i)
 > un número que dice que tan adaptado
 este (inversamente proporcional al costo)
 $1/\text{costo}$ o $1/\sqrt{\text{costo}}$

seleccion: ruleta, torneo

crucía

$[v_{i1}, \dots, v_{in}]$	$[v_{j1}, \dots, v_{jn}]$
$[v_{k1}, v_{k+1}, \dots, v_{kn}]$	

mutación: seleccionar algunos valores
 y cambiarlos aleatoriamente.
 prob.

12 mayo 2026

Busquedas Locales

class BusquedaLocal():

def costo(self, x):

return f(x)

def vecinos(self, x):

return generador de vecinos de x

def valor_aleatorio(self):

return x

14 mayo 2026

$f(x)$ $\hat{=}$ encontrar x^* tal que $f(x^*) \leq f(x) \forall x$

encontrar $x = (x_1, \dots, x_n)$ tal que se cumplan restricciones.

Tengo un valor $x = (x_1, \dots, x_n)$
y x_i es una variable que pertenece al dominio i que es un conjunto $\{v_{i,1}, \dots, v_{i,k_i}\}$

y aparte tengo una serie de restricciones que me digan cuantas se cumplen y cuantas no. Tres tipos:

$R_j(x_i \neq v_i) \in \{True, False\}$ Restricción binaria

$R(x_i = v_i, x_j = v_j) \in \{True, False\}$ Restricción binaria

$R(x_i = v_i, x_j = v_j, \dots) \in \{True, False\}$
Restricción

Problem

encontrar $x_i = y_i \quad \forall \quad x_i = x$

tal que todas las restricciones se cumplan.

S	E	N	D	
+	M	O	R	E
M	O	N	E	Y